

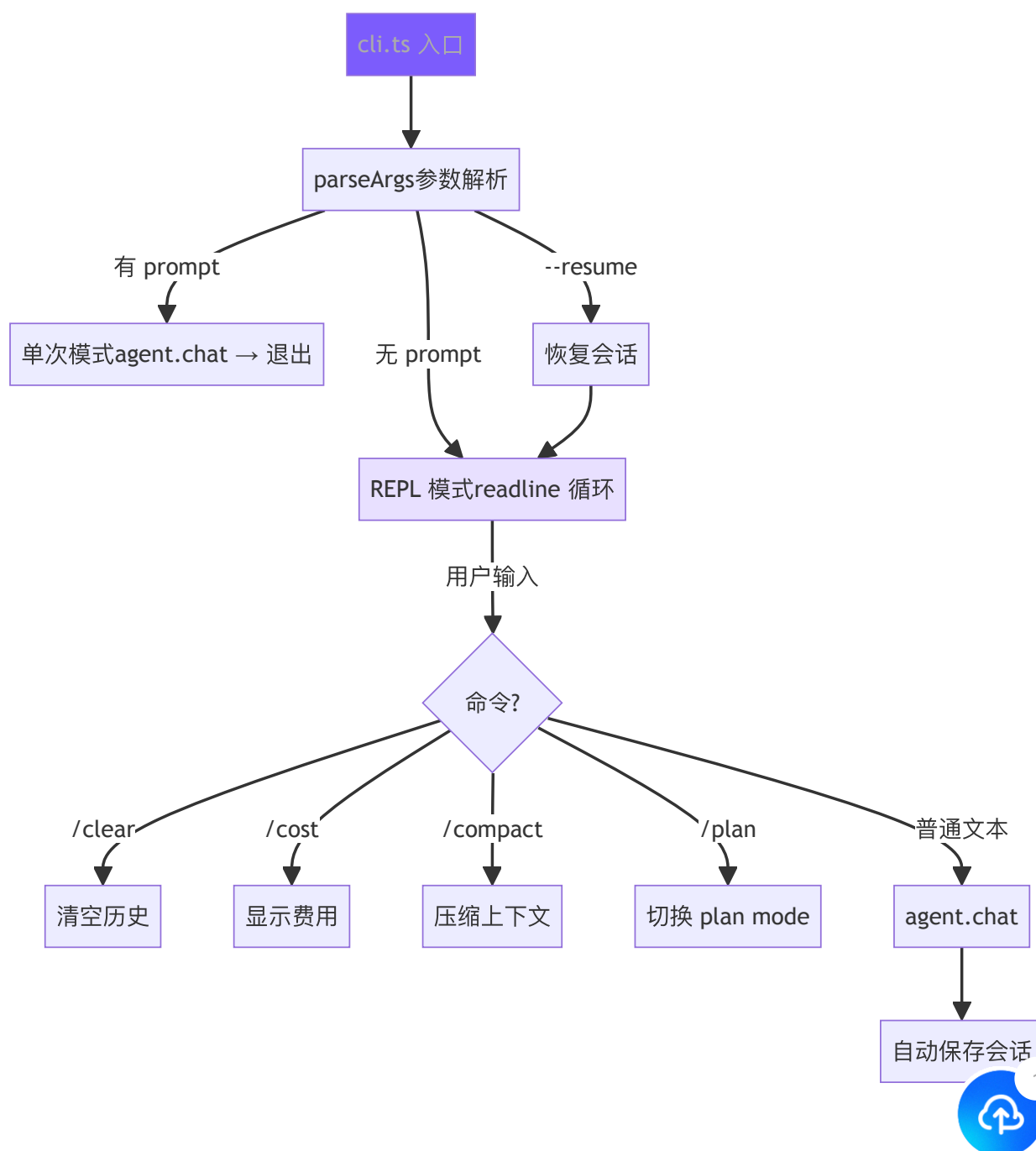


4. CLI 与会话

本章目标

到上一章为止，agent 的内核已经齐了——循环、工具、System Prompt——可它还没有一张能跟人对话的脸。这一章造用户接口。

一个命令行入口负责解析参数（`--yolo`、`--resume` 这些开关），一个 REPL 让人一句一句地聊，`Ctrl+C` 能中途打断当前这轮，聊完的对话写到磁盘、下次 `--resume` 接着聊。



► 跑这一章: `node steps/run.mjs 4` (无需 API key)。它会存  English
`--resume` 接着聊。加 `--diff` 看它比上一章多了什么。想拿自己的 prompt 连真实模型, 就加 `--live` (读 `.env` 里的 key, `--py` 跑 Python 版)。

我们的实现

上一章的 agent 每次都从空白开始——进程一关, 聊过的全忘了。这一章给它加上会话: 每轮把消息数组存到磁盘, `--resume` 时读回来接着聊。相对上一章, 新增了一个 `session.ts`, `cli.ts` 也长出了 `--resume` 和 `/clear`:

TypeScript

Python

diff

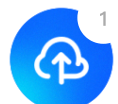
```
@@ -2,4 +2,5 @@ import * as readline from "readline";
import { pathToFileURL } from "url";
import { Agent } from "./agent.js";
+import { saveSession, loadSession } from "./session.js";

// A tiny REPL: read a line, hand it to the agent, repeat. One-shot
@@ -13,4 +14,11 @@ export async function runCli(argv: string[] = process.argv) {

    const agent = new Agent();
+    // --resume: reload the saved conversation before doing anything
+    const resume = argv.includes("--resume");
+    argv = argv.filter((a) => a !== "--resume");
+    if (resume) {
+        const saved = loadSession();
+        if (saved) { agent.loadHistory(saved as any); console.log(`(loaded ${saved.length} messages)`); }
+    }

    const oneShot = argv.join(" ").trim();
@@ -18,4 +26,5 @@ export async function runCli(argv: string[] = process.argv) {
    const input = oneShot;
    await agent.chat(input);
+    saveSession(agent.history());
    return;
}

@@ -28,5 +37,7 @@ export async function runCli(argv: string[] = process.argv) {
```



```

const input = line.trim();
if (input === "exit" || input === "quit", );
+ if (input === "/clear") { agent.clearHistory(); saveSess:
+ if (input) await agent.chat(input);
if (input) saveSession(agent.history());
ask();
});

```

会话本身很朴素——整段对话本来就是消息数组，存盘就是把它写成 JSON：

TypeScript

Python

```

typescript
export function saveSession(messages: unknown[]): void {
  try { writeFileSync(SESSION_FILE, JSON.stringify(messages, null,
  }

export function loadSession(): unknown[] | null {
  if (!existsSync(SESSION_FILE)) return null;
  try { return JSON.parse(readFileSync(SESSION_FILE, "utf-8")); }
  }

```

跑一下：先记住一件事，关掉，再 `--resume`，它还记得：

```

$ node steps/run.mjs 4
▶ step 4 demo (no API key – local mock model)  sandbox: <sandbox>
$ mini-claude Remember that my favorite color is blue.
$ mini-claude --resume What is my favorite color?

```

```

Got it – your favorite color is blue.
(resumed 2 messages)
Your favorite color is blue.

```

参数解析



TypeScript

Python

 English

typescript

```
// cli.ts - parseArgs

function parseArgs(): ParsedArgs {
  const args = process.argv.slice(2);
  let permissionMode: PermissionMode = "default";
  let thinking = false;
  let model = process.env.MINI_CLAUDE_MODEL || "claude-opus-4-6";
  let apiBase: string | undefined;
  let resume = false;
  let maxCost: number | undefined;
  let maxTurns: number | undefined;
  const positional: string[] = [];

  for (let i = 0; i < args.length; i++) {
    if (args[i] === "--yolo" || args[i] === "-y") {
      permissionMode = "bypassPermissions";
    } else if (args[i] === "--plan") {
      permissionMode = "plan";
    } else if (args[i] === "--accept-edits") {
      permissionMode = "acceptEdits";
    } else if (args[i] === "--dont-ask") {
      permissionMode = "dontAsk";
    } else if (args[i] === "--thinking") {
      thinking = true;
    } else if (args[i] === "--model" || args[i] === "-m") {
      model = args[++i] || model;
    } else if (args[i] === "--api-base") {
      apiBase = args[++i];
    } else if (args[i] === "--resume") {
      resume = true;
    } else if (args[i] === "--max-cost") {
      const v = parseFloat(args[++i]);
      if (!isNaN(v)) maxCost = v;
    } else if (args[i] === "--max-turns") {
      const v = parseInt(args[++i], 10);
      if (!isNaN(v)) maxTurns = v;
    } else if (args[i] === "--help" || args[i] === "-h") {
      console.log(`Usage: mini-claude [options] [prompt] ...`);
      process.exit(0);
    }
  }
}
```



```

    } else {
      positional.push(args[i]);
    }
  }

  return {
    permissionMode, model, apiBase, resume, thinking, maxCost, max
    prompt: positional.length > 0 ? positional.join(" ") : undefined
  };
}

```

 English

TypeScript 版手写循环而不用 commander.js，因为只有 11 个参数，零依赖更轻。用 `for` 而不是 `forEach` 是因为带值参数（`--model claude-sonnet`）需要 `++i` 跳到下一个元素。Python 直接用标准库 `argparse`。

两种运行模式

TypeScript

Python

```

// cli.ts - main

async function main() {
  const { permissionMode, model, apiBase, prompt, resume, thinking

  // API key 从环境变量获取，不支持命令行传递（避免泄露到 shell history）
  // 优先级: OPENAI_API_KEY + OPENAI_BASE_URL → ANTHROPIC_API_KEY →
  const resolvedApiKey = resolveApiKey(apiBase);
  if (!resolvedApiKey) {
    printError(`API key is required. Set ANTHROPIC_API_KEY or OPENAI_API_KEY`);
    process.exit(1);
  }

  const agent = new Agent({ permissionMode, model, apiBase, apiKey

  if (resume) {
    const sessionId = getLatestSessionId();
    if (sessionId) {
      const session = loadSession(sessionId);

```

typescript



```
    if (session) agent.restoreSession(session);
  }
}

if (prompt) {
  await agent.chat(prompt);      // 单次模式：执行后退出
} else {
  await runRepl(agent);          // REPL 模式：交互循环
}
}
```

 English

REPL 实现

TypeScript

Python

typescript

```
// cli.ts - runRepl

async function runRepl(agent: Agent) {
  const rl = readline.createInterface({ input: process.stdin, out: process.stdout });

  let sigintCount = 0;
  process.on("SIGINT", () => {
    if (agent.isProcessing) {
      agent.abort();
      console.log("\n (interrupted)");
      sigintCount = 0;
      printUserPrompt();
    } else {
      sigintCount++;
      if (sigintCount >= 2) { console.log("\nBye!\n"); process.exit(0); }
      console.log("\n Press Ctrl+C again to exit.");
      printUserPrompt();
    }
  });

  printWelcome();
}
```

// rl.once 而非 rl.on: 保证严格串行, 避免多个 chat 并发修改消息历史



```

const askQuestion = (): void => {
  printUserPrompt();
  rl.once("line", async (line) => {
    const input = line.trim();
    sigintCount = 0;

    if (!input) { askQuestion(); return; }
    if (input === "exit" || input === "quit") { console.log("\n");
      askQuestion(); return; }

    if (input === "/clear") { agent.clearHistory(); askQuestion(); }
    if (input === "/cost") { agent.showCost(); askQuestion(); }
    if (input === "/compact") {
      try { await agent.compact(); } catch (e: any) { printError(e); }
      askQuestion(); return;
    }
    if (input === "/plan") { agent.togglePlanMode(); askQuestion(); }

    try {
      await agent.chat(input);
    } catch (e: any) {
      if (e.name !== "AbortError" && !e.message?.includes("abort")) {
        printError(e);
      }
      askQuestion();
    }
  });
};

askQuestion();
}

```

Ctrl+C 的双重语义：处理中按下 → 中断当前操作，回到输入提示；空闲时按下 → 第一次提醒，第二次退出。这避免了两种意外：手滑 Ctrl+C 导致整个会话丢失，以及 Agent 跑偏时只能眼睁睁等它跑完。

**** rl.once vs rl.on **：**rl.on 注册的 handler 不会等 await agent.chat() 完成就响应下一行输入，导致多个 chat 并发修改消息历史。rl.once 每次只监听一行，处理完再递归注册，天然串行。Python 的 while + input() + await 没有这个问题。

会话持久化



TypeScript

Python

 English

typescript

```
// session.ts

const SESSION_DIR = join(homedir(), ".mini-claude", "sessions");

export function saveSession(id: string, data: SessionData): void {
  ensureDir();
  writeFileSync(join(SESSION_DIR, `${id}.json`), JSON.stringify(data));
}

export function getLatestSessionId(): string | null {
  const sessions = listSessions();
  if (sessions.length === 0) return null;
  sessions.sort((a, b) => new Date(b.startTime).getTime() - new Date(a.startTime).getTime());
  return sessions[0].id;
}
```

每次 `agent.chat()` 完成后自动保存，保存失败静默忽略（不能因为磁盘满让整个对话崩溃）。恢复时直接把消息数组加载回 Agent：

TypeScript

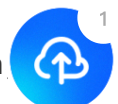
Python

typescript

```
// agent.ts

private autoSave() {
  try {
    saveSession(this.sessionId, {
      metadata: { id: this.sessionId, model: this.model, cwd: process.cwd(),
        startTime: this.sessionStartTime, messageCount: this.messages.length },
      anthropicMessages: this.useOpenAI ? undefined : this.anthropicMessages,
      openaiMessages: this.useOpenAI ? this.openaiMessages : undefined
    });
  } catch {}
}

restoreSession(data: { anthropicMessages?: any[]; openaiMessages?: any[] }) {
  if (data.anthropicMessages) this.anthropicMessages = data.anthropicMessages;
  if (data.openaiMessages) this.openaiMessages = data.openaiMessages;
}
```




```
if (data.openaiMessages) this.openaiMessages = data.openaiMessages;
printInfo(`Session restored (${this.getMessageCount()} messages)`);
}
```

 English

终端 UI — ui.ts

所有输出通过 `ui.ts` 统一格式化：

TypeScript

Python

typescript

```
// ui.ts (使用 chalk)
```

```
export function printToolCall(name: string, input: Record<string,
const icon = getToolIcon(name);      // read_file → 📖, run_she
const summary = getToolSummary(name, input);
console.log(chalk.yellow(`\n ${icon} ${name}`) + chalk.gray(` `);
}
```

```
export function printToolResult(name: string, result: string) {
const maxLen = 500;
const truncated = result.length > maxLen
  ? result.slice(0, maxLen) + chalk.gray(`\n ... (${result.length}
  : result;
console.log(chalk.dim(truncated.split("\n").map((l) => "  " + l)
}
```

工具结果在 UI 层截断到 500 字符——这是给人看的显示，完整结果已在消息历史中。

真实 Claude Code 比这多做了什么

我们的界面是一个 readline 加几行打印。Claude Code 的界面是一整套跑在终端里的 UI 框架——差距全在「让它在真实终端里稳、好用、崩不坏」这些地方。



Claude Code 的入口是 `src/entrypoints/cli.tsx` ——用 React/Ink 把组件模型搬进终端，支持流式 Markdown 渲染、Vim 模式、多 Tab、键盘自定义。会话追加写入，崩溃安全。



English

终端原生 vs GUI

这是一个主动选择。开发者的工作流在终端里，打开浏览器意味着上下文切换。终端原生就是另一个命令行工具，跟 `git`、`grep` 一样嵌入到已有工作流。具体好处：SSH 环境可用、可接管道 (`echo "fix" | claude`)、支持 `tmux` 多实例并行、内存开销接近零。

React/Ink 的作用是弥补终端的交互限制——有了组件模型，流式输出、diff 视图这类复杂 UI 才变得可维护。

可观察的自主性

Claude Code UX 的核心理念：**Agent 自由行动，但让用户实时看到每一步。**

```
📖 read_file src/app.ts
1 | import express from ...
... (1234 chars total)

✎ edit_file src/app.ts
- const port = 3000
+ const port = process.env.PORT
```

中断成本远低于撤销成本。用户在 Agent 走错方向前 3 秒就能按 `Ctrl+C`，而不是等 20 秒执行完再花更多时间撤销。每个工具有 4 种渲染方法（开始/完成/被拒/报错），长时间运行的工具实时流式输出 `stdout`，而不是等完成才展示。

JSONL 会话存储

整体 JSON 覆盖写入有两个问题：写入中途崩溃会损坏整个文件；对话越长每次保存越慢。

JSONL 每轮追加一行， $O(1)$ 写入，崩溃最多丢最后一行。文件系统的 `append` 操作通常是原子的。恢复时逐行解析，跳过末尾不完整的行即可。



下一章：让 agent 的输出实时显示——流式输出与双后端支持。

 English

< 上一篇

3. System Prompt 工程

下一篇 >

5. 流式输出与双后端

